Library build with task selection: does repairing where programs disagree build a better library?

Date: 2026-10-14 onwards. One RTX 5090. Model: `claude-sonnet-5` for every writer and repair agent. Figure: `figs/` (made by `make_figures.py`). Code: `scripts/taskselect/library_loop.py`.

Summary

- A skill library is built from nothing by repairing one failing task per round. Two ways to choose the task are compared, with paired repetitions: by **disagreement** between programs (`lex`), and **at random among unsolved tasks** (`unsolved`). Each round writes 27 programs (9 tasks x 3) and runs them on 30 seeds; each arm makes 5 repairs.
- **Noise is large.** Five independent generations from the same empty library have SD 0.027 in mean pass rate, so two single runs differ by SD 0.038 and a gap below 0.075 cannot be shown with two repetitions (0.061 with three, 0.043 with six).
- The runs are in progress; results are not part of this report yet.

Proposition and hypotheses

Setting. A language model writes a Python program for a robot task from a skill library (markdown files of rules and helpers). When programs fail, an agent repairs one task, and the lesson is written back into the library. The library is the only thing that carries knowledge from one round to the next. Repairing is expensive (about an hour of agent and simulator time per task), so a build repairs only a few tasks. **Which task to repair next decides what the library learns.**

Proposition. The most informative task to repair is the one where the current programs **disagree**: some pass and some fail on the same seeds, many unsolved tasks fail at the same stage, and the failures have varied traces. Disagreement says that the task is neither hopeless nor already solved, and that a fix at a shared failure stage will carry over to other tasks. Repairing by this rule (`lex`) should build a better library than repairing a random unsolved task (`unsolved`) at the same number of repairs.

Hypotheses (all about the library after the same number of repairs, same tasks, same writers):

id	statement	measure	how it can fail
H1 (primary)	the final library of `lex` gives a higher pass rate than that of `unsolved`	mean pass rate over 27 programs (9 tasks x 3), and over 6 programs per task, on 30 seeds	gap within the noise, or negative
H2	`lex` improves a library faster along the rounds	gain over round 0, mean over rounds 1 to 5	no gain difference
H3	`lex` lessons transfer: tasks that were not repaired improve	per-task passes, unrepaired tasks	gains only on repaired tasks
H0 (noise)	one run's mean moves by 0.03 (SD) with no change in the library, so an effect smaller than the detectable gap cannot be shown	five round-0 generations from the empty library	(this is a measured fact, not a claim)

What can be concluded. The smallest gap that two arms can reliably show (5% level, 80% power) is 0.106 with 1 repetition, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6. A result below that is reported as "not shown", not as "no effect".

Method: the library, the loop and the update rule

The library. Markdown files in `.claude/libero/skills/` (`grasp`, `localize`, `manipulation`, `transport`, `README`). Writers read them as context; programs never import them. A build starts from the empty library `L00`, and each repair produces the next one (`L01`, `L02`, ...). Every version is stored in `outputs/<run>/library/Lnn`.

Algorithm.

```
L <- empty library
repeat 5 times:
    P <- 3 fresh programs per task, written from L          # 9 tasks
    run P on seeds 51..80                                  # 810 trials
    U <- tasks where no program passes every seed
    t <- pick(U)                                           # the only difference between arms
    findings <- repair(t)                                  # up to 5 tries, dev seeds 51..65
    L <- L + lessons(findings)                             # add rules, never remove
final: 3 fresh programs per task from L, run on seeds 51..80

pick(U), arm "unsolved":  a random task of U
```

```
pick(U), arm "lex":       the task of U with the largest (split, shares, trace), compared in that order
    split  = how much programs disagree per seed (entropy of how many pass)
    shares = how many tasks of U fail at the same stage
    trace  = how varied the failure traces are
```

A trial's failure stage comes from its API trace only: `crash`, `perceive` (object not found), `no_motion`, `no_close`, `air` (closed on nothing), `slip` (held, then lost), `held_end`, `released` (placed wrong). `LEX.md` has a worked example.

Repair. The agent reads the passing and failing traces of the task's 3 programs, then writes up to 5 task-level programs (no seed branches, no hard-coded coordinates), each run on the 15 dev seeds and diagnosed from its failed trials. The kept program is the one with the most passes (ties: the later one). It writes `findings.md`: root causes, pass counts and "Generalizable Patterns".

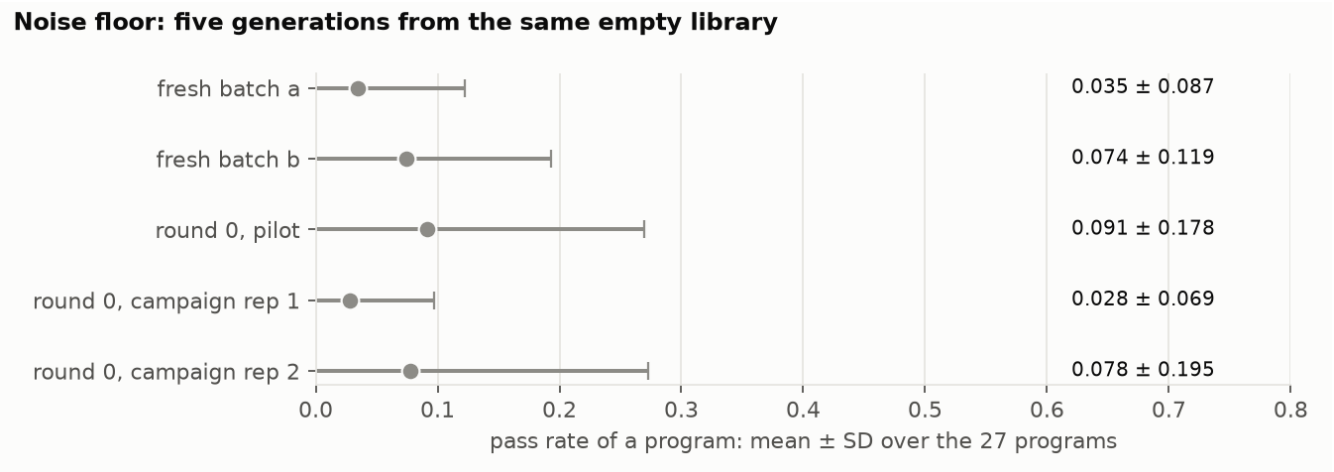
Library update rule (promotion). After the repair, the coordinator reads `findings.md`, edits the skill files and logs it in a ledger (`skill_promotions.jsonl`: files changed, reason). Rules in force:

1. A lesson is added as a short rule with its trigger and evidence, in the file of its category (grasp, localise, place, carry). Generic patterns are preferred over task facts.
2. No rule is ever removed or corrected: the library only grows (4.7 KB after the first repair, 17.6 KB after five in one run).
3. The kept program's pass count on the dev seeds is the only evidence required; there is no test on other seeds.
4. Skills may use the allowed APIs only (no simulator state, no `.bddl`/`.xml` reads).

Protocol and controls.

item	value
pool	9 swap tasks: goal (3), spatial (3), object (3); 21 more swap tasks are held out and unused so far
probe seeds	51-80 (30), every round; repair development seeds are 51-65
repetition	round 0 is written once and shared by both arms; arm order alternates; 5 repetitions planned
same in both arms	model (`claude-sonnet-5`), K, writers' prompts, repair budget, seeds, API facts note
differs	only the selection rule
noise control	five independent round-0 generations from the empty library, and a re-measurement of each final library with 3 more programs per task

1. Noise floor



Independent generations of programs from the same empty library, same pool, same seeds (27 programs, 9 tasks x 3, each program run on its seeds). The plot shows the mean and SD over the 27 programs of each program's pass rate, for five generations. Most programs pass nothing and a few pass most seeds, so the SD is large against the mean.

generation	mean pass rate	note
fresh batch a	0.035	15 seeds
fresh batch b	0.074	15 seeds
round 0 of the second pilot repetition	0.091	15 seeds
round 0 of campaign rep 1	0.028	30 seeds
round 0 of campaign rep 2	0.078	30 seeds
mean, SD between generations	**0.062, 0.027**	

- One run's mean moves by 0.027 (SD) with no change to the library. The difference of two single runs has SD 0.038.
- **Smallest gap between two arms that is reliably seen** (5% level, 80% power): 0.106 with 1 repetition per arm, 0.075 with 2, 0.061 with 3, 0.053 with 4, 0.043 with 6.
- **Seeds are not the limit.** With 30 seeds the sampling noise of a run's mean is only 0.005 to 0.008. The noise comes from the programs the writers happen to produce and from the repairs, so the lever is more repetitions or more programs per task.
- Execution adds a little: re-running identical programs on the same seeds changed 11 of 15 per-seed outcomes for one program.

2. Deviations and limits

- **A first pilot is not reported.** Its round 0 had 9 programs patched by hand for API errors (missing `import numpy`, a list passed to the point prompt, a dict passed as a mask), the point-prompt misuse recurred in later rounds (up to 43% crashed trials in one arm), and it used 15 seeds and 3 repairs. It showed the pipeline runs and that the noise is large, and it led to the API-facts note in the writers' prompts. A second pilot repetition (15 seeds, 3 repairs, no patches) is used only for its round-0 generation in the noise floor. All results above come from runs without hand patches.
- **Molmo is off**, as in the upstream TC1 runs; writers are told not to use it.
- **Interrupted runs.** One arm's coordinator session was stopped by mistake and one hit the subscription rate limit for hours; both were restarted from the files on disk with a resume notice. The work done is unchanged.
- Probe seeds 51-65 are also the repair development seeds, so the repaired task is slightly favoured on half of its seeds in both arms.

Where things are, by experiment id

All paths are under `aspire/sim/outputs/` (gitignored). The live directory `.claude/libero/skills/` is only a working copy: each run saves its own libraries, so it is restored to the committed version when no run is active.

experiment id	what it is	libraries	probes and selections	repairs
`library_noise_a`, `library_noise_b`	fresh round-0 generations from the empty library	`library/L00`	`round_00/probe.json`	none
`library_lex_r2`, `library_unsolved_r2`	a pilot repetition (15 seeds, 3 repairs; shared round 0)	`library/L00` to `L03`	same	same
`library_lex_s30_r<k>`, `library_unsolved_s30_r<k>`	campaign (k = 1 to 5); `outputs/remeasure/<run>` holds the 3 extra programs per task and their probe (30 seeds 51-80, 5 repairs, 6 probes; shared round 0 per repetition)	`library/L00` to `L05`	same	same
`skill_snapshots/family_start`	skills after the first single-task fix loop; equals the committed `.claude/libero/skills`	5 files	none	none

Each run also has `state.json` (pool, rounds, settings) and `coordinator.md` (the exact prompt it was started with). In each repetition the second arm's round 0 is a byte-identical copy of the shared one (`probe.json` compared with `cmp`).